DECK 6 OF 12 · FRC JAVA CRASH COURSE

Arm PID: move it, hold it

Control theory and Java for a single-jointed arm on a NEO

WPILib 2026 · Java · Command-based

OVERVIEW

What this deck covers

Theory

What P, D, I and feedforward each do to an arm.

Tuning

A safe order: kG, then kS if needed, then kP, then kD.

Java

An arm subsystem on the roboRIO, then with a motion profile.

CONTROL THEORY

Open loop and closed loop

Open loop: no measurement

The code picks an output and sends it.

Nothing measures the result.

A change in battery, friction or load goes uncorrected.

Closed loop: measure and correct

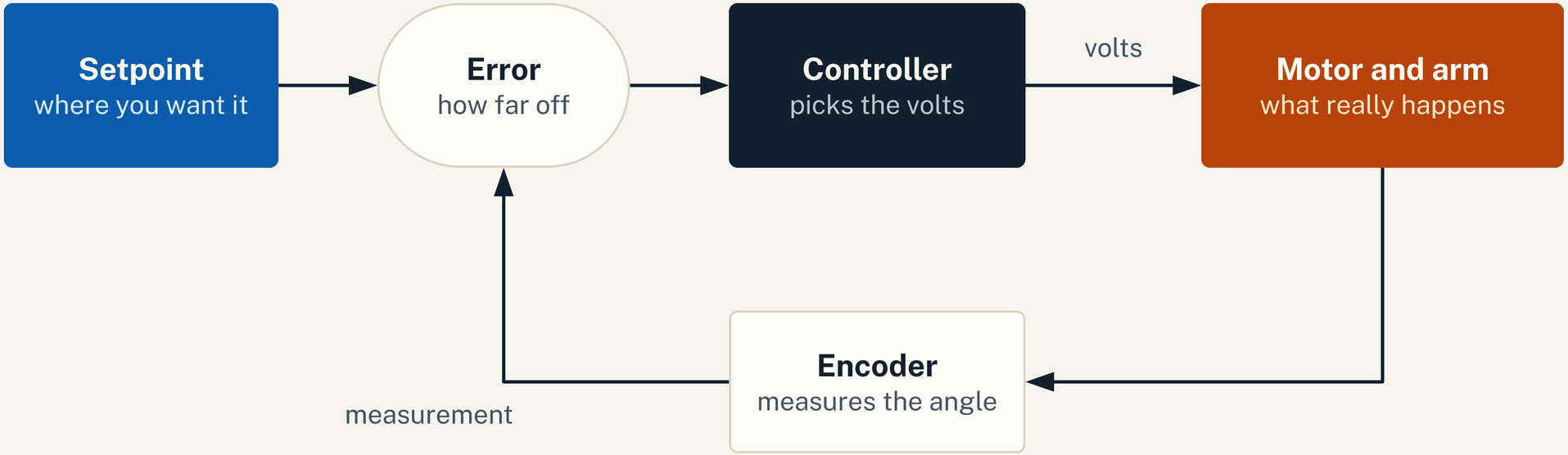
The code sends an output and reads a sensor.

It compares the reading with the target and corrects.

That measurement is called **feedback**.

CONTROL THEORY

The feedback loop



Error = setpoint - measurement. Everything in this deck is about what to do with that one number.

CONTROL THEORY

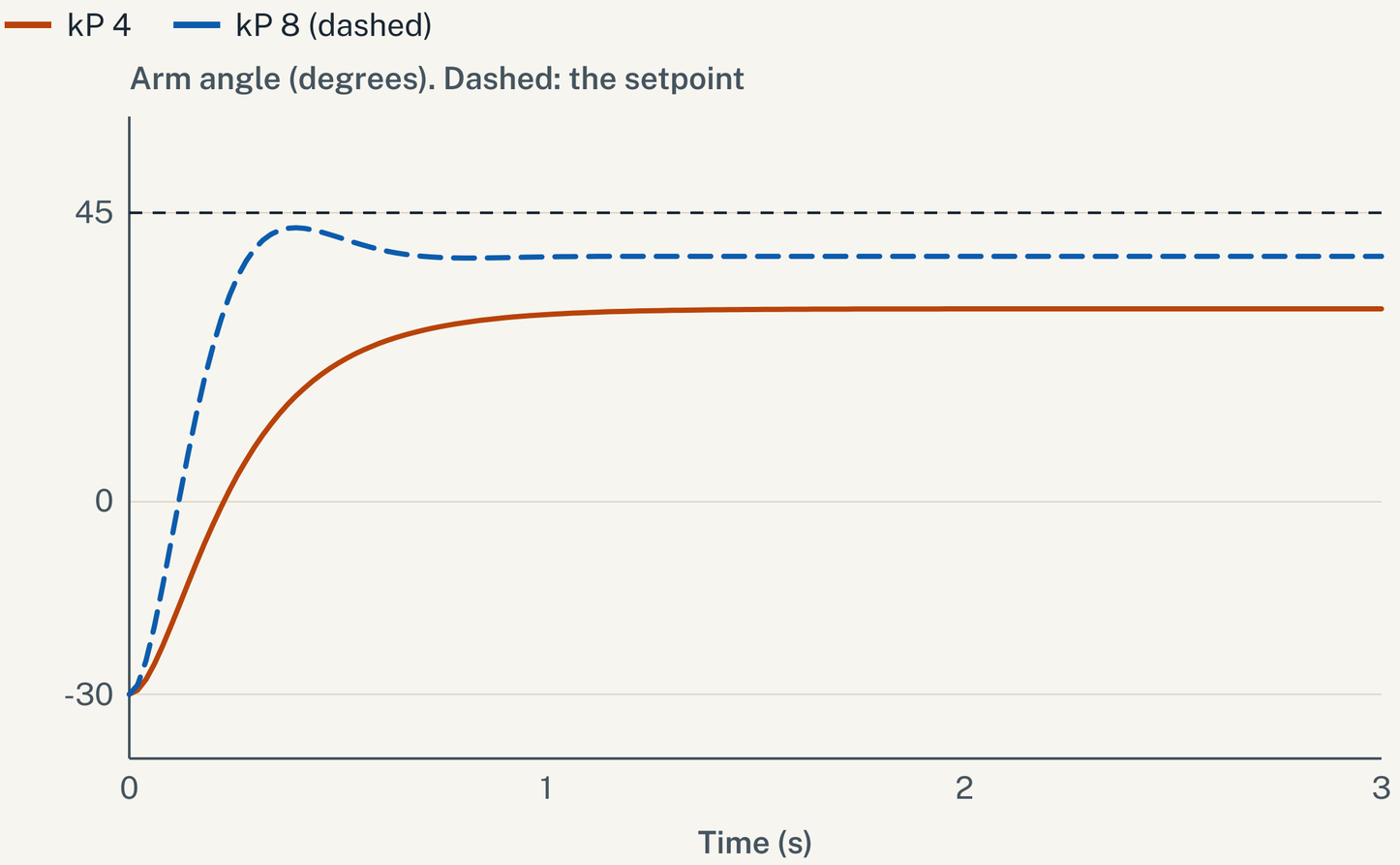
One example arm for the whole deck

Part	Our numbers	What it sets
Arm	Uniform bar, 0.8 m long, 4 kg	Inertia 0.85 kg m^2
Gearbox	60:1	Trades speed for torque
Motor	One NEO: 12 V, 2.6 N·m and 105 A at stall, 5676 RPM free	How much push you get
Sensor	Through-bore encoder on the arm shaft	Knows the angle at power-on
Loop	New volts every 20 ms, reading one loop old	How late every decision is
Travel	Rests on a stop at -30°, top stop at 110°. 0° is flat	Where gravity pulls hardest

Every plot simulates this arm: DC motor, gearing, inertia and gravity, stepped at the 20 ms loop.

CONTROL THEORY

P: push harder the farther away you are



1 The rule

volts = kP × error. More error, more volts.

2 The sag

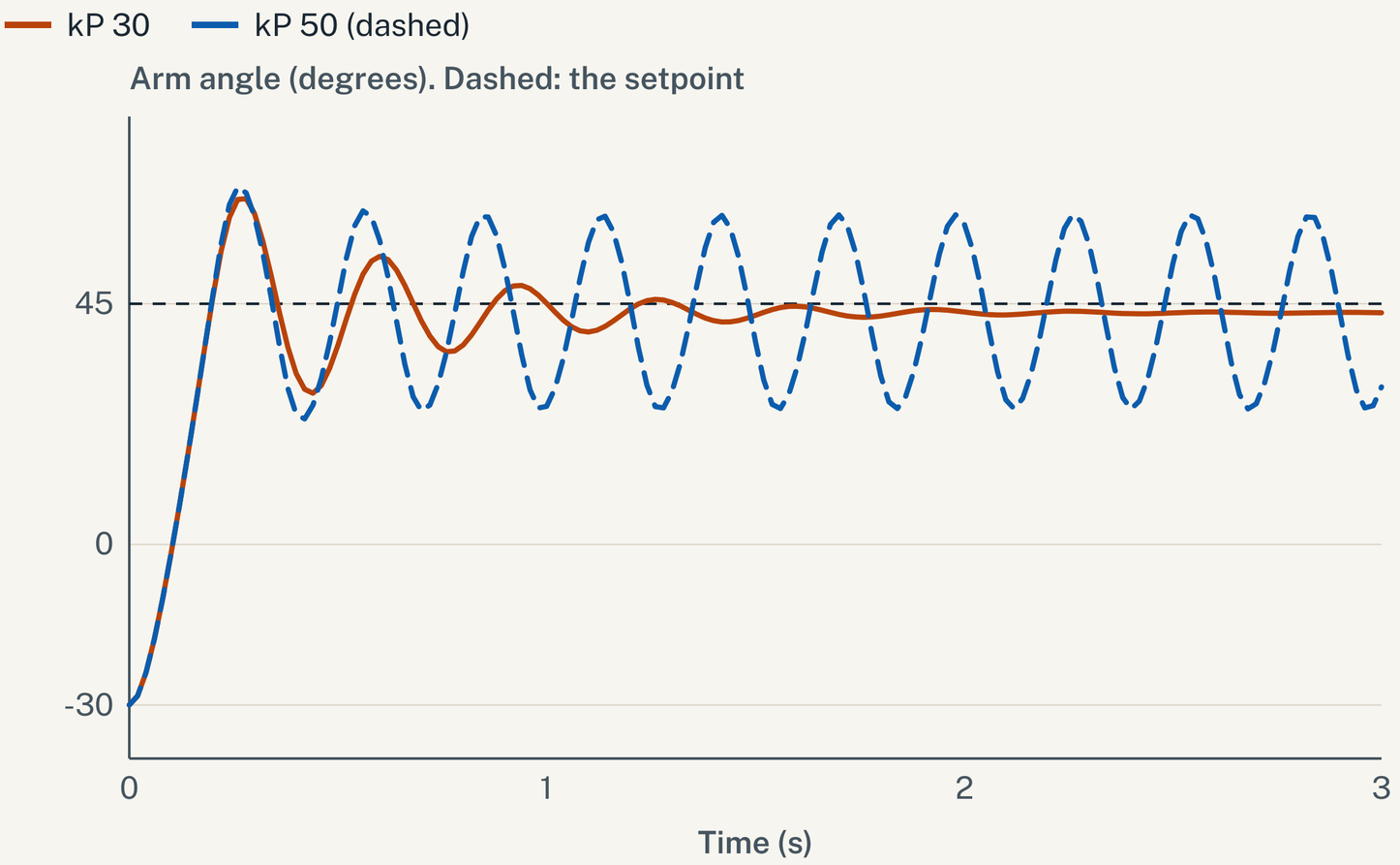
kP 8 stops at 38.2°, 6.8° short. kP 4 is 15.0° short.

3 Why it stays

Holding there takes 0.95 V. P only makes volts from error, so some error has to stay.

CONTROL THEORY

Too much P: overshoot, then shaking



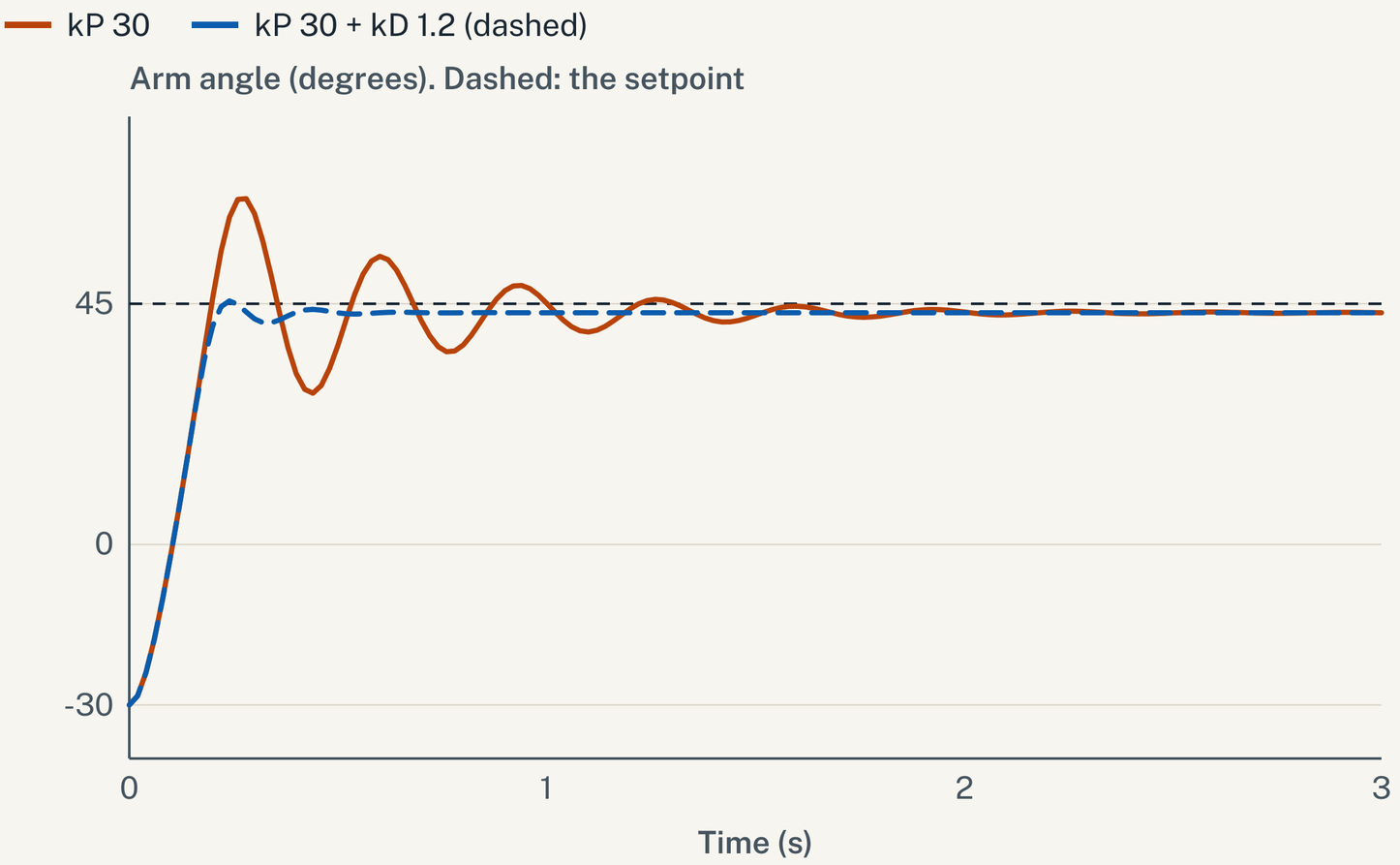
1. **Less sag**
   kP 30 ends 1.7° short, so that worked.

2. **New problem**
   It swings to 64.6°, 19.6° past the setpoint.

3. **Too far**
   kP 50 never settles. It is still swinging 36° wide after 3 s.

CONTROL THEORY

D: reducing overshoot



1 The rule

volts = kD × how fast the error changes.

2 When it acts

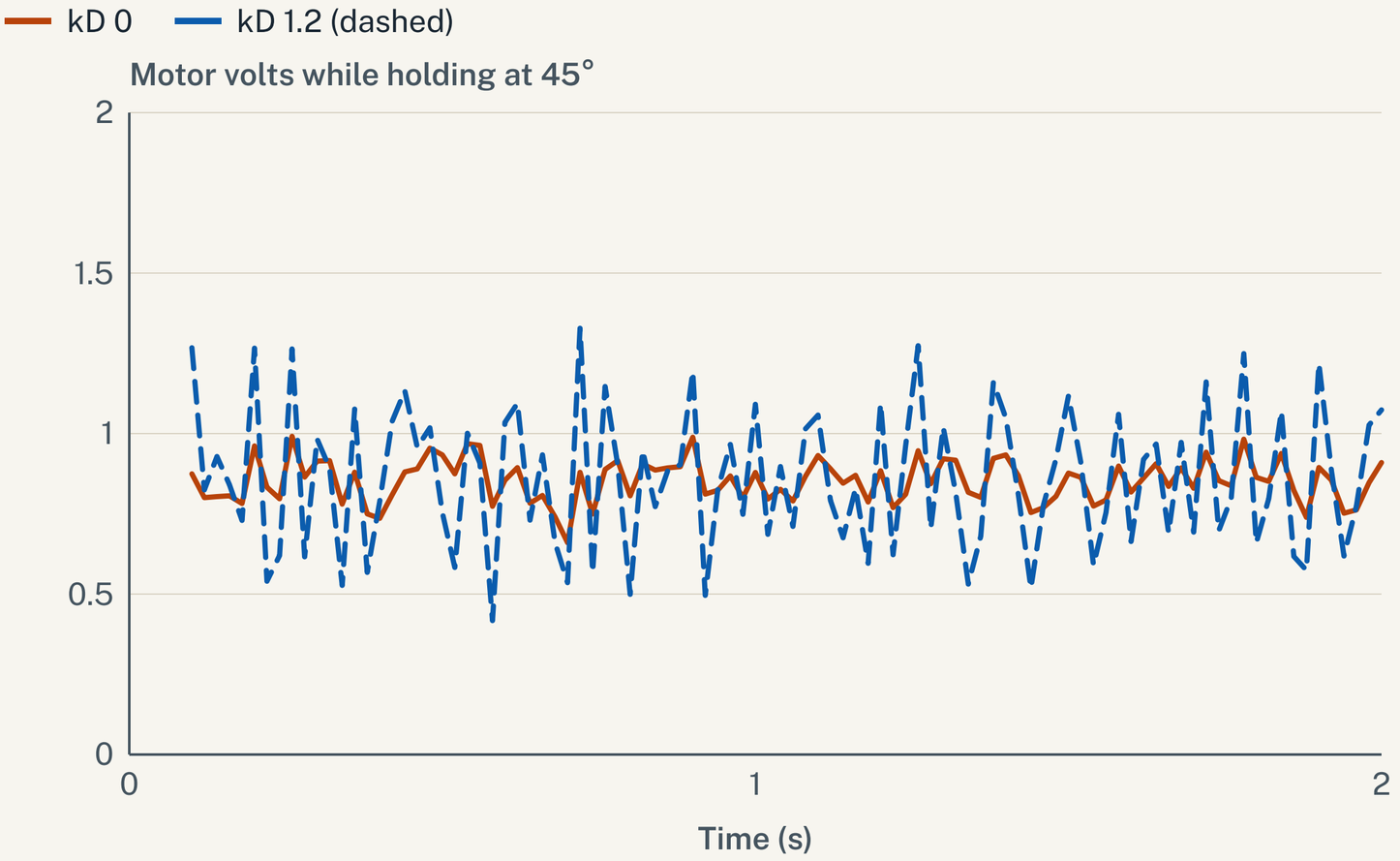
D subtracts volts while the arm closes in, before it reaches the setpoint.

3 What changed

Overshoot drops from 19.6° to 0.5°. The arm still ends 1.7° short.

CONTROL THEORY

D multiplies sensor noise



1 Why

D divides a tiny change by 0.02 s. That is the same as multiplying it by 50.

2 The test

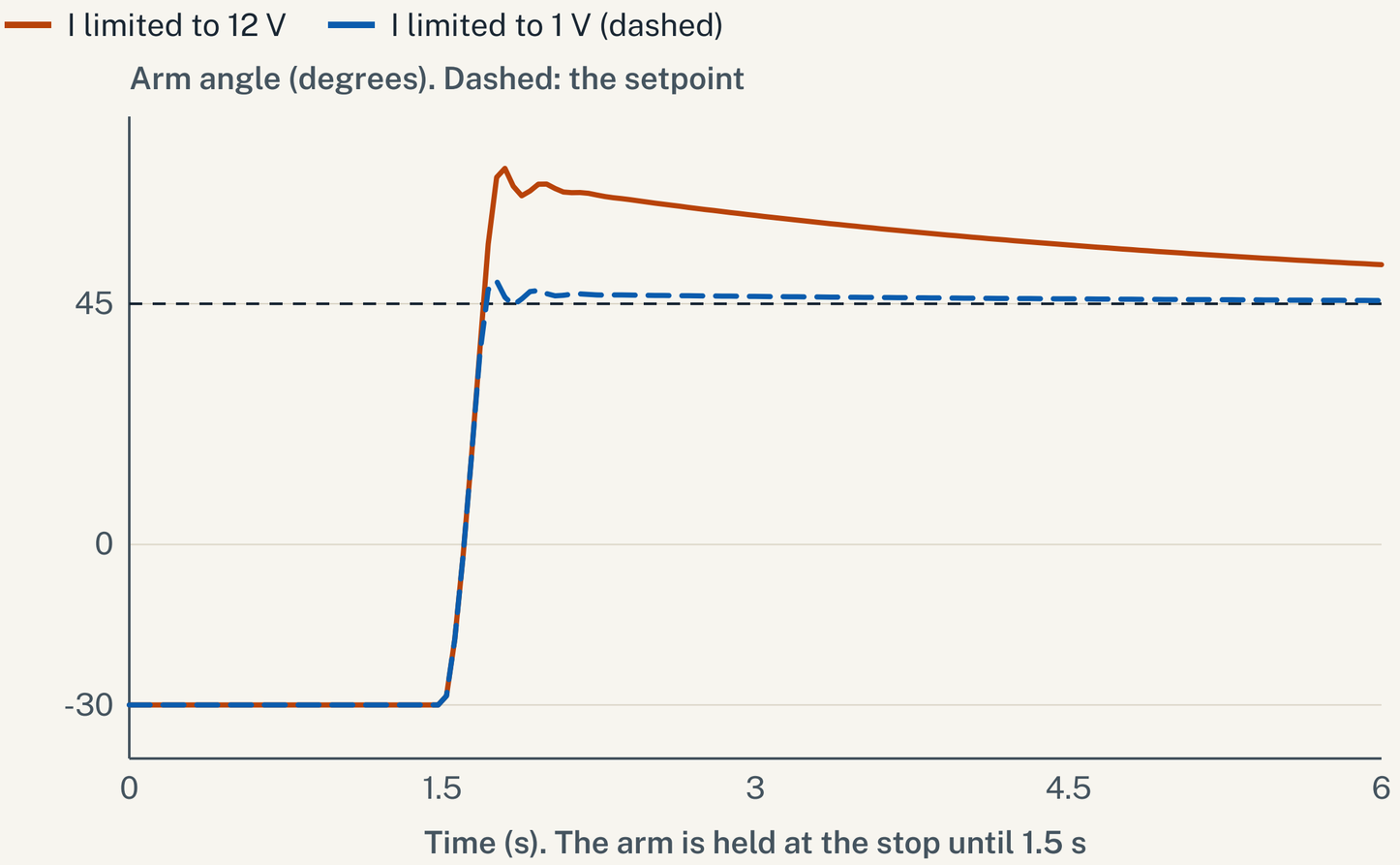
We added ±0.2° of fake noise to the reading. The arm itself moved 0.31°.

3 The result

Volts spread 0.33 V with no D and 0.91 V with it. Use as little D as does the job.

CONTROL THEORY

I: the slow fix that can wind up



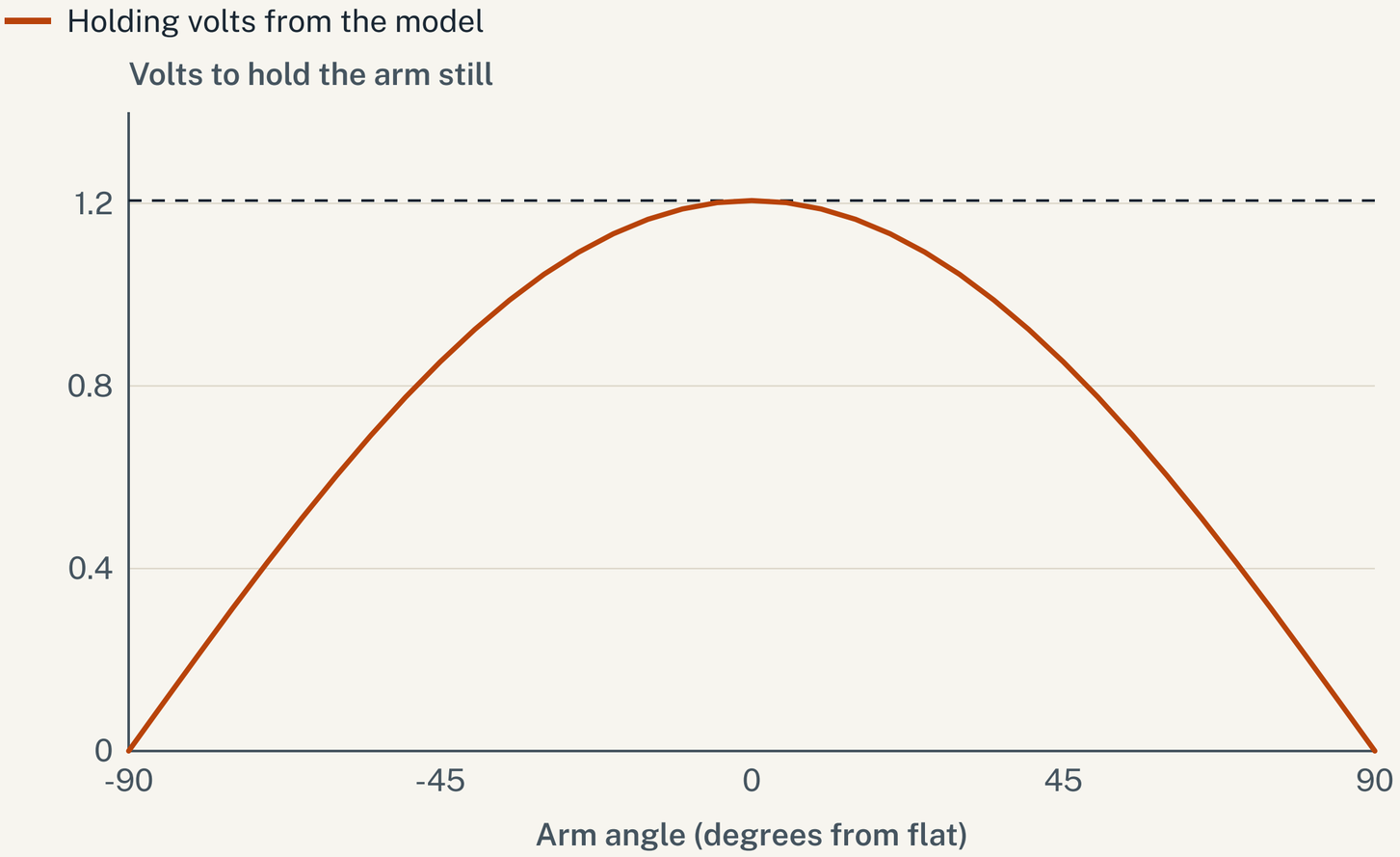
1 The idea
I adds up error over time. kI 8 here.

2 Windup
Stuck for 1.5 s, the total grows. Let go and it swings to 70.6°.

3 Slow recovery
4.5 s after release it is still 7.8° high. WPILib limits I to ±1 by default.

CONTROL THEORY

Gravity: volts to hold the arm at each angle



1 Flat is hardest
The bar sticks straight out, so gravity's lever is longest: 1.21 V.

2 The cosine
At 45° it takes 0.85 V. Straight up it takes 0 V. Your calculator's cos does this.

3 Why 0° is flat
Then cos(0) = 1, and kG simply means "volts to hold it flat".

CONTROL THEORY

Where 1.21 volts comes from

1. **Gravity torque when flat: 15.7 N·m**
   Mass × g × distance to the middle: 4 × 9.8 × 0.4.

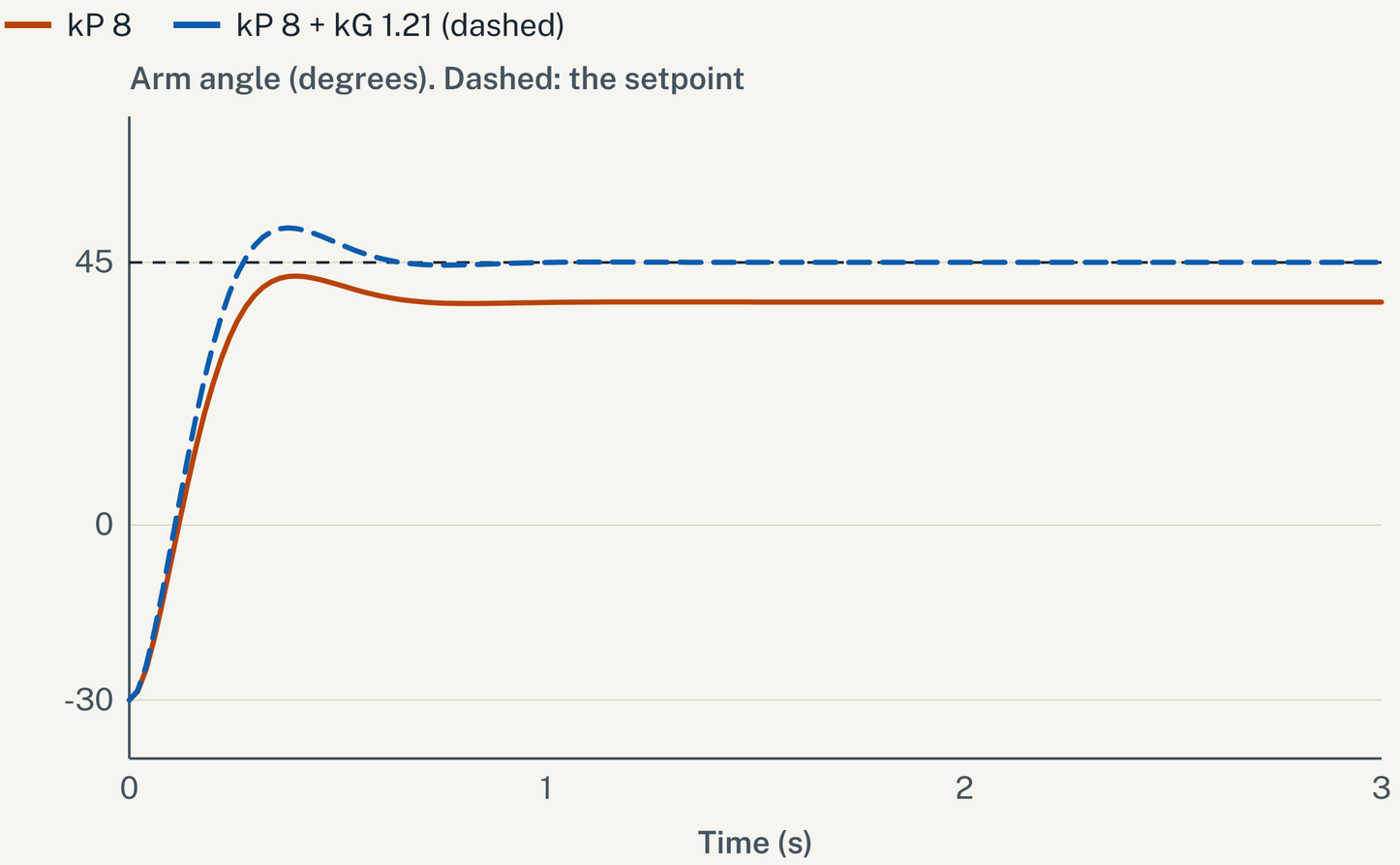
2. **Torque at the NEO: 0.261 N·m**
   The gearbox divides it by 60.

3. **Current for that torque: 10.6 A**
   Divide by the NEO’s 0.0248 N·m per amp.

4. **Volts to push that current: 1.21 V**
   Multiply by the NEO’s 0.114 ohms. The motor is not turning, so it makes no voltage of its own.

CONTROL THEORY

Feedforward: add the volts you already know



1 **The rule**
volts = kG × cos(angle), added before PID runs.

2 **Same gentle P**
kP 8 alone ends at 38.2°. With kG it ends at 45.0°, after a 5.9° overshoot.

3 **Why it works**
The volts go in before any error appears.

Feedforward does the work. PID cleans up.

Feedforward is the volts you can predict. PID handles only what you could not.

CONTROL THEORY

The rest of feedforward: kS and kV

ArmFeedforward(kS, kG, kV) adds all three. You already know the middle one.

kS: friction

The volts needed to overcome friction and start moving.

kG: gravity

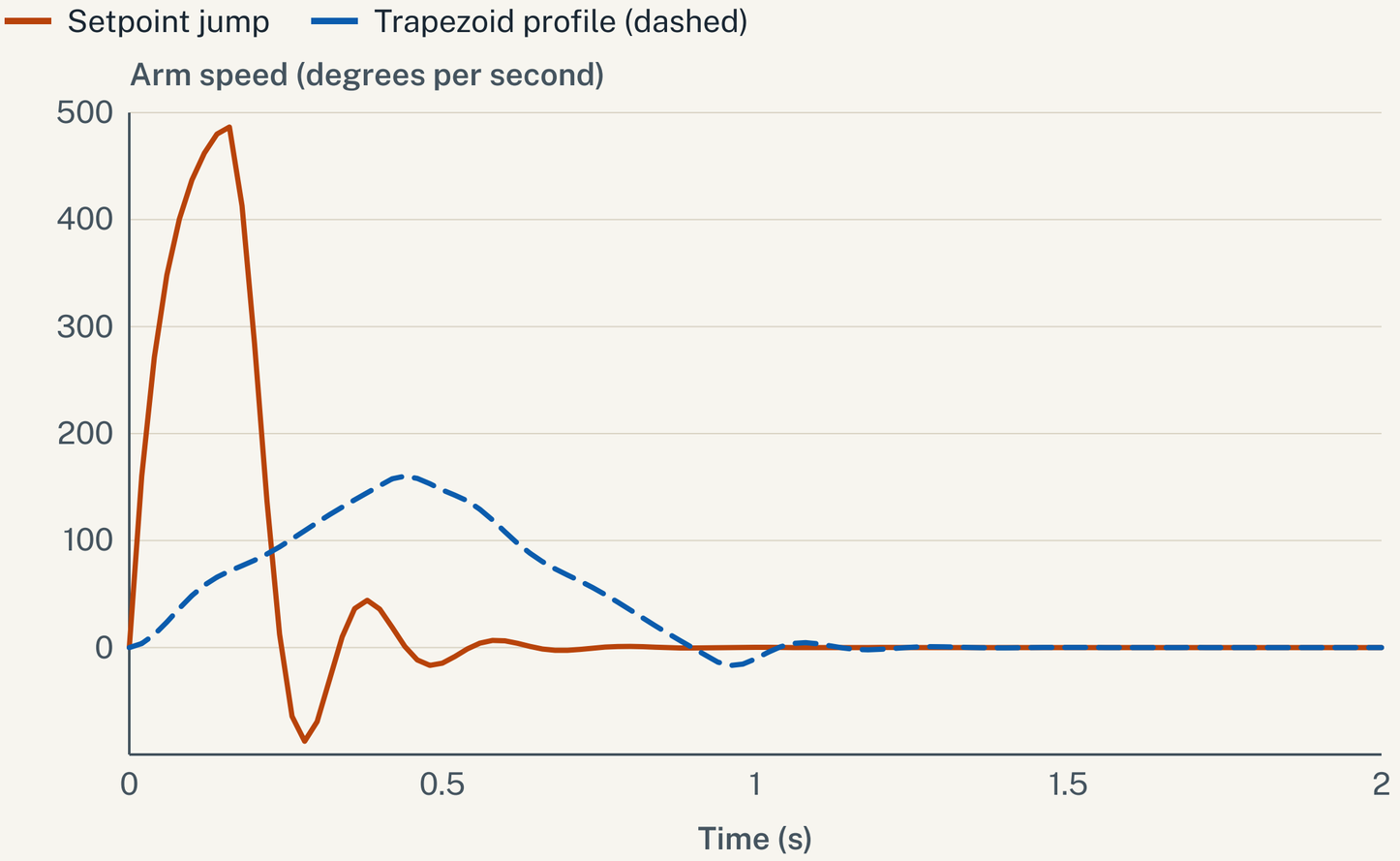
1.21 V × cos(angle) on our arm.

kV: speed

Back-EMF grows with speed. Our arm takes 1.19 V per rad/s of speed.

CONTROL THEORY

Motion profiles: do not jump the setpoint



1. **The jump**
   12 V and 105 A at once. Hard on chain, gears and battery.

2. **The trapezoid**
   Speed up, cruise, slow down. 4.8 V and 17 A at most.

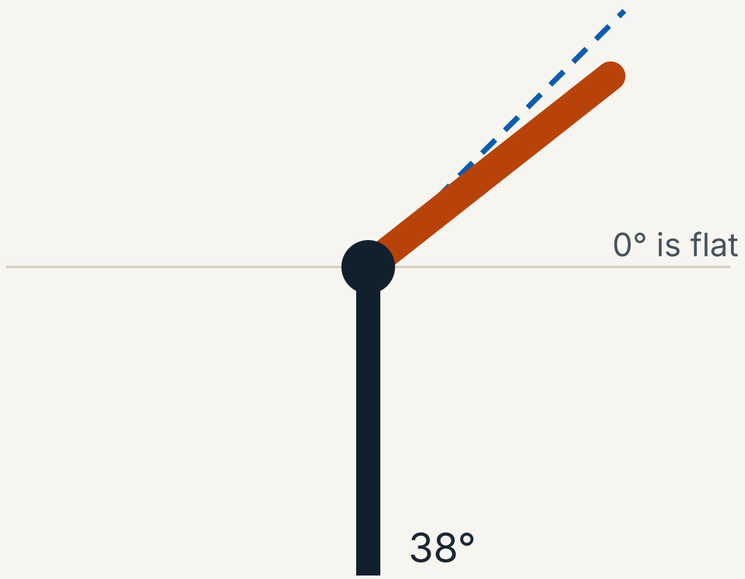
3. **Who did the work**
   Over this move, feedforward supplied 88% of the volts.

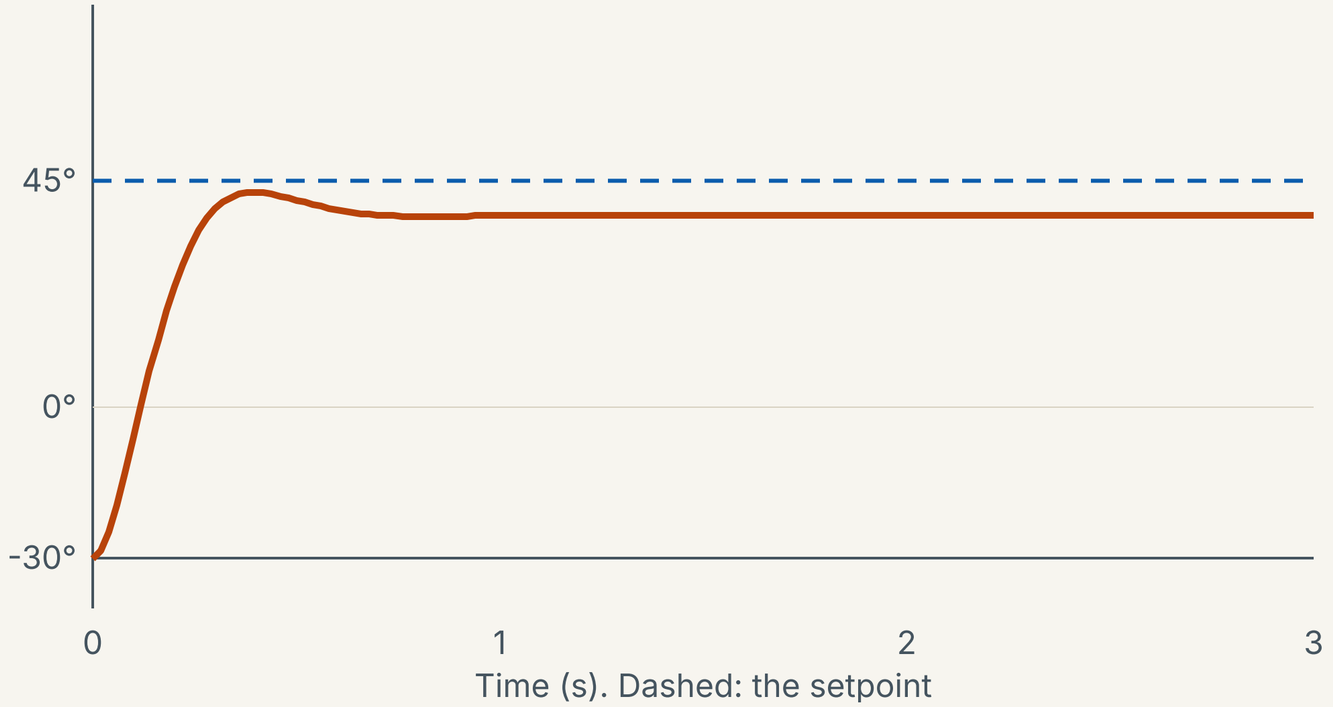
Five gain sets on one arm

1 of 5 **P only**

kP 8

Stops 6.8° short





CONTROL THEORY

Tuning steps, in order

1. **kG first**
   All gains at 0. Raise kG until the arm stays where you leave it.

2. **kS, only if needed**
   If the arm sticks and then jumps, add a small kS.

3. **Raise kP**
   Start low and raise it in steps. Stop when it starts to overshoot.

4. **Add kD**
   Add D until the overshoot goes. If the motor buzzes, back off.

5. **Test with small, safe moves**
   Try 10° before 90°. Hand on the disable key, every time.

EXERCISE

Test gains in the arm playground

1. Scan the Arm playground QR on the last slide, or open docs/arm-playground/index.html from the repo.

2. Press **P only**. Write down how far short the arm stops.

3. Press **Too much P**, then slide kP down until the shaking stops.

4. Find your own gains. Turn on the profile if you want it.

Done when: the status line reads **Settled** and you can read your gains as Java constants.

Now the Java

The same arm, written two ways

BEST PRACTICE

One unit in Java, convert at the boundary

> **Pick one unit for all your Java: radians. Convert once, where the code meets the hardware.**

The SPARK reports rotations, so ArmUnits is the only class that converts. Hold the arm flat and save the raw reading as ENCODER_FLAT_ROTATIONS.

```
public static double encoderToArmRad(double encoderRotations) {
  double turnsFromFlat = encoderRotations - ENCODER_FLAT_ROTATIONS;
  return MathUtil.angleModulus(Units.rotationsToRadians(turnsFromFlat));
}
public static double armRadToMotorRotations(double armRad) {
  return Units.radiansToRotations(armRad) * GEAR_RATIO;
}
```


2027: REVLib alphas drop conversion factors too. Native units carry over.

PROGRAMMING

The loop on the roboRIO

ArmSubsystem.java

```
// A NEO is brushless. kBrushed would damage it.
private final SparkMax motor = new SparkMax(MOTOR_ID, MotorType.kBrushless);
private final SparkAbsoluteEncoder throughBore = motor.getAbsoluteEncoder();
private final PIDController pid = new PIDController(KP, 0, KD);
private final ArmFeedforward feedforward = new ArmFeedforward(KS, KG, KV);

/** Runs every 20 ms. Feedforward holds, PID fixes the rest. */
private void driveToTarget() {
  double angle = getAngleRad();
  double ffVolts = feedforward.calculate(angle, 0.0);
  double pidVolts = pid.calculate(angle, targetRad);
  double volts = ffVolts + pidVolts;
  lastVolts = MathUtil.clamp(volts, -MAX_VOLTS, MAX_VOLTS);
  motor.setVoltage(lastVolts);
}
```

PROGRAMMING

Command factories: moveTo and holdPosition

ArmSubsystem.java

```
  /** Near the goal for 0.1 s in a row. */
1 public final Trigger atGoal =
1     new Trigger(pid::atSetpoint).debounce(0.1);

  public Command moveTo(double goalRad) {
2   double goal =
2       MathUtil.clamp(goalRad, SOFT_MIN_RAD, SOFT_MAX_RAD);
3   return startRun(() -> startMove(goal), this::driveToTarget)
        .until(atGoal)
        .withName("Arm moveTo");
  }

4 public Command holdPosition() {
4   return startRun(this::startHold, this::driveToTarget)
        .withName("Arm hold");
  }
```

1. **atGoal**
   A Trigger that needs 0.1 s near the goal, so one lucky reading does not count.

2. **Clamp the goal**
   A bad setpoint gets stopped here, before the motor sees it.

3. **Start clean**
   Sync the encoders, reset PID, then loop until atGoal.

4. **Hold**
   Keep the last goal if the arm got there. If not, hold where it is.

PROGRAMMING

A default command that holds, and triggers

RobotContainer.java

```
private void configureBindings() {
  // With nothing else running, hold still.
  // This also stops the arm jumping when you enable.
  arm.setDefaultCommand(arm.holdPosition());

  driver.a().onTrue(arm.moveTo(STOW_RAD));
  driver.y().onTrue(arm.moveTo(SCORE_RAD));

  // A Trigger can watch the arm, not just a button.
  // This only fires if the arm has arrived.
  driver.rightBumper().and(arm.atGoal).onTrue(Commands.print("Score!"));
}
```

The bumper fires only when the driver presses it **and** the arm has arrived.

PROGRAMMING

Add a profile with ProfiledPIDController

ProfiledArmSubsystem.java

```
  private final ProfiledPIDController controller =
      new ProfiledPIDController(
          KP, 0, KD,
1         new TrapezoidProfile.Constraints(
1             MAX_VELOCITY_RAD_PER_S, MAX_ACCEL_RAD_PER_S2));

  private void driveToTarget() {
    double angle = getAngleRad();
    // Moves the profile one step, then runs PID against it.
2   double pidVolts = controller.calculate(angle, targetRad);
3   double speed = controller.getSetpoint().velocity;
3   double ffVolts = feedforward.calculate(angle, speed);
    double volts = ffVolts + pidVolts;
    lastVolts = MathUtil.clamp(volts, -MAX_VOLTS, MAX_VOLTS);
    motor.setVoltage(lastVolts);
  }
```

1. **The trapezoid**
   Top speed 2.5 rad/s, acceleration 6 rad/s^2.

2. **One step**
   Each loop, the setpoint moves a little toward the goal.

3. **kV gets a speed**
   The profile knows the target speed, so feedforward can use it.

BEST PRACTICE

Safety goes in before tuning

> **Add soft limits, a current limit and an output clamp first. Keep people clear and a hand on the disable key.**

Start with low gains. On the Driver Station, Enter disables and Space is the emergency stop.

```
// NEO rotations, because the NEO's encoder is the selected sensor.
config.softLimit
    .forwardSoftLimit(armRadToMotorRotations(SOFT_MAX_RAD))
    .forwardSoftLimitEnabled(true)
    .reverseSoftLimit(armRadToMotorRotations(SOFT_MIN_RAD))
    .reverseSoftLimitEnabled(true);
```

PROGRAMMING

Tune without risking hardware

ArmSimModel.java

```
private final SingleJointedArmSim physics =
    new SingleJointedArmSim(
        DCMotor.getNEO(1),
        GEAR_RATIO,
        SingleJointedArmSim.estimateMOI(ARM_LENGTH_METERS, ARM_MASS_KG),
        ARM_LENGTH_METERS,
        HARD_STOP_MIN_RAD,
        HARD_STOP_MAX_RAD,
        true, // simulate gravity
        HARD_STOP_MIN_RAD); // start on the low stop
private final Mechanism2d mechanism = new Mechanism2d(2, 2);
private final MechanismLigament2d armLine =
    new MechanismLigament2d("Arm", ARM_LENGTH_METERS, 0);
```

update() feeds the SPARK's volts in and the angle back out. SmartDashboard.putData("Arm", mechanism) publishes the drawing.

2027: SmartDashboard is removed. Telemetry replaces putData.

EXERCISE

Tune the arm in the simulator

1. In Constants.java, set KP to 2 and KD, KG, KV to 0. Run **Simulate Robot Code**.

2. Enable, press Y, and watch the arm in AdvantageScope or the sim GUI. It sags.

3. Follow the tuning steps: set KG, then KV, then raise KP, then add KD.

Done when: Y moves the arm to 45° and it holds within 2°, with no shaking.

COMMON MISTAKE

The arm runs away at full speed

What you see	You enable, and the arm slams to a stop as fast as it can go.
Why it happens	Motor and encoder disagree on direction. Moving “toward” the goal makes the error bigger, so P pushes harder.
The fix	Robot disabled: lift the arm by hand. The angle must go up. Then at about 1 V: the arm must rise. Flip ENCODER_INVERTED or MOTOR_INVERTED.

COMMON MISTAKES

Five more gotchas, by symptom

What you see	Why it happens	The fix
Ask for 45 and the arm drives to its top limit	Wrong units: 45 radians is about 7 turns	Use Units.degreesToRadians
Arm leans on a stop and the motor gets hot	Setpoint out of range, so error never reaches 0	Clamp goals to the soft limits
Always off by the same ratio	Gear ratio wrong in ArmUnits	Turn the arm 90° by hand and check
Correct at first, off after a restart	Relative encoder, no homing: it reads 0 at power-on	Use the absolute encoder, or home on a stop
Arm sags more with kG, 3.5° not 1.7°	kG sign backwards: up is negative in your code	Make up positive and keep kG positive

And I windup: if the arm overshoots after being stuck, cap I with setIntegratorRange or leave it out.

NEXT SEASON

What changes for 2027

In 2026	Changing for 2027
roboRIO runs the 20 ms loop	Systemcore replaces the roboRIO
Java 17	Java 25
edu.wpi.first imports	org.wpilib imports
SmartDashboard.putData for Mechanism2d	Telemetry and Tunables replace it
REVLib 2026 still has conversion factors	REVLib alphas drop them: use native units

The theory does not change. Gravity still follows a cosine. All 2027 details are alpha and may change.

RECAP

What this deck covered

- Why P alone sags, and why too much P shakes.
- D reduces overshoot and multiplies noise.
- Feedforward for what you can predict: kG × cos(angle), with 0° at flat.
- Tuning in order with small, safe moves: kG, kS, kP, kD.
- An arm subsystem with a motion profile.

Next: Deck 7 compares REV and CTRE motors and what they can do on board.

REFERENCE

Official docs



Tuning a vertical arm

docs.wpilib.org



Trapezoidal profiles

docs.wpilib.org



SPARK feedforward

docs.revrobotics.com



Arm playground

Interactive arm simulation